

软件工程基础第 8 章—软件测试
学习笔记（精炼版）

中南大学计算机学院 任胜兵

2026 年 6 月 1 日

目录

1	软件测试基础 (Software Testing Fundamentals)	2
2	代码复审 (Code Review)	3
3	白盒测试 (White-Box Testing)	4
3.1	逻辑覆盖法 (Logic Coverage)	4
3.2	基本路径覆盖法 (Basis Path Coverage)	5
3.3	循环覆盖法 (Loop Coverage)	6
4	黑盒测试 (Black-Box Testing)	6
4.1	等价分类法 (Equivalence Partitioning)	7
4.2	边界值分析法 (Boundary Value Analysis)	7
4.3	猜错法 (Error Guessing)	8
4.4	因果图法 (Cause-Effect Graphing)	8
4.5	黑盒测试综合策略	9
5	单元测试 (Unit Testing)	9
6	集成测试 (Integration Testing)	10
7	确认测试 (Validation Testing)	11
8	系统测试 (System Testing)	12
9	调试 (Debugging)	13
10	小结 (Summary)	14

1 软件测试基础 (Software Testing Fundamentals)

[p.2-19]

软件验证 (Software Verification)

[p.2]

软件验证是通过检查和提供客观证据表明软件已经满足规定需求的活动，贯穿软件整个生存周期。三种方式：

- **静态测试**（评审）：不运行程序，对软件进行分析和检查活动
- **动态测试**（测试）：通过运行软件检验动态行为和运行结果正确性
- **证明**：通过形式化的数学方法确保软件正确性

测试定义与 Myers 观点

[p.7-8]

软件测试对象不仅仅是程序代码，而是**软件生存周期各阶段的文档和代码**。据统计，64% 的错误属于分析和设计错误，编码错误仅占 36%。[p.7]

G.J.Myers 测试观点（经典三论断） [p.8]：

1. 测试是为了**寻找错误**而运行程序的过程
2. 一个好的测试用例在于能发现**至今未发现的错误**
3. 一个成功的测试是发现了**至今未发现的错误的测试**

核心定义：软件测试就是试图以最少的代价，发现软件分析、设计和编码中存在的各种不同类型的错误，从而提高软件质量，降低软件成本。[p.8]

测试原则

[p.9]

1. **尽早和不断地测试**——错误发现越晚，改正代价越大
2. **严格执行测试计划**——较早确定测试计划并严格执行
3. **注意错误群集现象**——应用 **Pareto 原则**（80% 错误集中在 20% 模块）
4. **测试规模从小到大**——单元测试 → 集成测试 → 系统测试
5. **由独立的第三方进行测试**——避免开发者思维定式
6. **保证测试用例的完整性和有效性**
7. **保存所有测试用例和出错统计直至软件淘汰**

测试工具

[p.10-16]

按工作方式分：静态分析工具、动态测试工具。[p.10]

按功能分：测试计划工具、测试设计与开发工具、测试执行工具、测试评价工具、测试管理工具、辅助测试工具。[p.10]

静态分析工具：扫描程序正文，分析数据流和控制流——变量检查、逻辑结构检查、接口检查、编程风格检查、静态特性统计。[p.11]

动态分析工具：有控制地运行程序，监视记录运行情况——语句执行次数统计、CPU 执行开销估算、执行时间分析、软硬件资源利用分析。[p.12]

测试组织

[p.17]

- 独立测试之前，软件开发者负责模块单元测试，确保每个模块完成详细设计功能
- 开发者也常进行集成测试，确保模块按概要设计要求形成系统
- 系统形成后，**独立测试小组**介入，确保系统满足需求分析和用户意图

测试与调试的区别

[p.18]

维度	测试 (Testing)	调试 (Debugging)
目标	发现错误症状	定位错误原因并改正
性质	查找错误的过程	纠错的过程
执行顺序	先于调试	后于测试
输入	测试用例 + 测试配置	错误症状 + 源码
输出	测试结果 + 错误率数据	修改后的软件配置

动态测试步骤——V 模型

[p.19]

软件测试与软件开发各阶段形成 **V 形对应关系**：需求分析 ↔ 系统测试，概要设计 ↔ 集成测试，详细设计 ↔ 单元测试，编码 → 源代码（测试输入）。每一开发阶段都有对应的测试级别，体现”尽早测试”原则。

2 代码复审 (Code Review)

[p.20-26]

概述

[p.20]

代码复审在程序通过编译和静态分析工具检查之后、动态测试之前进行。是一种人工测试方法，能够有效发现 30%–70% 的逻辑设计和编码错误。

三种复审方法对比

[p.24-26]

维度	代码会审 (Code Inspection)	走查 (Walkthrough)	办公桌检查 (Desk Checking)
参与人数	4 人左右 (组长 1+ 作者 1+ 程序员 1-2)	多人会议	1 人 (作者本人)
准备阶段	会前发放材料给与会者熟悉	会前发放材料, 指定人设计测试用例	无需分发
会议方式	作者逐句朗读讲解, 他人质疑讨论	与会者扮演计算机角色, 人工” 执行” 程序	作者独自分析检验
测试用例	对照检查表	预先设计测试用例, 人工” 输入” 运行	按错误检验表自查或仿走查
会后处理	问题清单交作者, 修改后可能再次会审	同代码会审	个人修改
特点	同行启发, 利学习交流	模拟执行, 偏重逻辑功能检查	最轻量级, 无组织开销

复审内容 [p.21-23]: 检查编码实现是否完备正确, 对照错误检验表查找结构、功能、编码标准和风格等方面的问题。

3 白盒测试 (White-Box Testing)

[p.27-41]

概念

[p.27]

白盒测试又称**结构测试**或**玻璃盒测试**, 以程序的内部逻辑结构为依据设计测试用例。目标是选取足够的测试用例对源代码实现充分覆盖。主要有三种方法: **逻辑覆盖法**、**基本路径覆盖法**、**循环覆盖法**。

3.1 逻辑覆盖法 (Logic Coverage)

[p.28-31]

按覆盖程度从弱到强分为五级标准:

语句覆盖 < 判定覆盖 < 条件覆盖 < 判定-条件覆盖 < 条件组合覆盖

覆盖标准	定义	英文
语句覆盖	每条可执行语句至少执行一次	Statement Coverage
判定覆盖	每个判定的真/假分支各至少执行一次（又称分支覆盖）	Decision Coverage / Branch Coverage
条件覆盖	每个判定的每个条件各至少取一次真和一次假	Condition Coverage
判定-条件覆盖	同时满足判定覆盖和条件覆盖	Decision/Condition Coverage
条件组合覆盖	每个判定中所有可能条件取值组合至少执行一次	Multiple Condition Coverage

注意：单一条件判定的条件覆盖即满足判定覆盖，但多条件判定则不一定。条件组合覆盖最强但测试用例数指数增长。[p.28]

逻辑覆盖设计步骤 [p.29]：

1. 选择逻辑覆盖程度类型
2. 选择测试路径以满足选定覆盖程度
3. 选择测试输入数据以满足选定测试路径和覆盖程度
4. 根据输入数据和路径计算预期结果

3.2 基本路径覆盖法 (Basis Path Coverage)

[p.32-37]

由 **T.J.McCabe**（麦凯伯）提出。**基本路径**是指程序中至少引进一条新语句或新条件的任一路径。循环在计算路径时只计算一次。[p.32]

主要步骤

[p.33-37]

1. **导出程序图：**以详细设计或源码为基础，将复合条件判定转化为单一条件判定。圆圈 = 条件/语句，箭头 = 控制流。[p.34]
2. **计算环形复杂度 $V(G)$ ：**三种等价计算公式 [p.35]
 - $V(G) = E - N + 2$ （ E 为边数， N 为节点数）
 - $V(G) = P + 1$ （ P 为判定节点数，每个判定是单一条件）
 - $V(G) = R$ （ R 为程序图中封闭区域数）
3. **确定基本路径集：**基本路径数 = 环形复杂度大小。每条新路径应包含至少一条新有向边（弧）。[p.36]
4. **生成测试用例：**使每条基本路径至少执行一次。[p.37]

关键结论：满足基本路径覆盖不一定满足条件组合覆盖，反之亦然。为实现充分覆盖，可设计同时满足两者的测试用例。[p.37]

3.3 循环覆盖法 (Loop Coverage)

[p.38-41]

逻辑覆盖和基本路径覆盖对循环只测试一次，不够充分。结构化程序的三种循环：[p.38]

简单循环测试策略

[p.39] 假设 n 为最大循环次数：

- 循环 0 次（跳过整个循环）
- 循环 1 次（只通过一次）
- 循环 2 次
- 循环 m 次 ($m < n$)
- 分别循环 $n-1$ 、 n 和 $n+1$ 次

嵌套循环测试策略

[p.40]

1. 从最内层循环开始，外层循环设为最小次数
2. 对最内层使用简单循环策略，为范围外/排除值增加其他测试
3. 由内向外构建下一层循环的测试，外层循环为最小次数，内层为”典型”次数

串接循环测试策略

[p.41]

- 各循环彼此**独立**：使用简单循环策略
- 循环**不独立**（第二个循环变量的初始值依赖外层第一个循环的当前值）：使用嵌套循环策略

白盒测试方法对比

维度	逻辑覆盖法	基本路径覆盖法	循环覆盖法
关注点	判定/条件的覆盖程度	独立路径的遍历	循环的不同执行次数
代表性	五级递进覆盖标准	McCabe 环形复杂度	简单/嵌套/串接循环策略
循环处理	仅循环一次	仅计算一次	专门细化处理
适用性	一般模块	逻辑复杂模块	循环密集模块

4 黑盒测试 (Black-Box Testing)

[p.42-65]

概念

[p.42]

黑盒测试又称**功能测试**或**数据驱动测试**，将测试对象视为黑盒子，不考虑内部逻辑结构，只依据规格说明书检查功能是否正常。白盒测试用于早期（单元测试），黑盒测试用于后期（确认测试、系统测试）。[p.42]

常发现的错误类型

[p.43]

- **功能错误**：功能不正确/遗漏/实现不该实现的功能
- **接口错误**：不能正确接收或输出信息
- **数据错误**：数据结构错误或外部信息访问错误
- **性能错误**：性能需求得不到满足
- **初始化或终止错误**：不能正确初始化或终止

4.1 等价分类法 (Equivalence Partitioning)

[p.45–51]

将所有可能的输入数据（有效或无效）划分为若干等价类，从每个等价类选一个“代表”形成测试用例。核心假定：**等价类中所有数据对于暴露错误的效力相同**。[p.45]

等价类划分指南

[p.46]

1. 规定了取值范围或值的个数：定义 **1 个有效等价类 + 2 个无效等价类**（小于下界、大于上界）
2. 规定为布尔量：1 个有效等价类 + 1 个无效等价类
3. 规定了输入值集合或“必须如何”条件：1 个有效 + 1 个无效
4. 规定了输入数据必须遵守的规则：1 个有效（符合规则）+ 若干无效（从不同角度违反规则）
5. 输入数据的不同值程序做不同处理：每个值为一个有效等价类 + 1 个无效等价类
6. 若等价类中各元素处理方式不同：应进一步划分为更小等价类

设计步骤

[p.47]

1. 划分等价类，形成等价类表
2. 为每个等价类规定唯一编号
3. 设计新测试用例，使其**尽量多地**覆盖尚未被覆盖的**有效等价类**，重复至全部有效等价类覆盖
4. 设计新测试用例，使其覆盖一个且仅一个**无效等价类**，重复至全部无效等价类覆盖

4.2 边界值分析法 (Boundary Value Analysis)

[p.52–54]

经验表明，程序在处理边界情况时最容易出错。边界值分析法是等价分类法的补充，重点选择正好等于、刚刚小于和刚刚大于边界的值。[p.52]

设计指南

[p.53]

- 输入范围 $[a, b]$ ：测试 a 、 b 、略小于 a 、略大于 b 的值
- 规定了值的个数：测试最大个数、最小个数、比最大多 1、比最小少 1
- 输出条件同样适用以上两条
- 有序集合：选取第一个和最后一个元素
- 预定义数据结构边界（如数组下标 0 到 99）：测试边界数据项

4.3 猜错法 (Error Guessing)

[p.55–56]

测试人员依靠经验和直觉，从各种方案中选出最可能引起程序出错的方案。能充分发挥经验优势，集思广益，在测试基础较差时尤为有效。[p.55]

示例——排序程序 [p.56]：除边界值分析（空序列、单元素、满序列）外，猜错法补充：已排好序、逆序排列、全部相等。

4.4 因果图法 (Cause-Effect Graphing)

[p.57–64]

适用于被测程序具有多种输入条件且输出依赖于输入条件的各种组合的情况。借助图形设计测试用例，将因果图转换为判定表。[p.57]

因果图符号

[p.58]

- 恒等 (Identity)：若原因存在，则结果存在
- 非 (NOT)：若原因存在，则结果不存在
- 与 (AND)：所有原因都存在，结果才存在
- 或 (OR)：任一原因存在，结果即存在

设计步骤

[p.59]

1. 分析规格说明，识别原因（输入条件/等价类）和结果（输出条件）
2. 分析语义和限制，找出因果关系和约束，画出因果图
3. 将因果图转换为判定表 (Decision Table)
4. 判定表的每一列写成一个测试用例

4.5 黑盒测试综合策略

[p.65]

1. 首先用边界值分析法设计测试用例
2. 必要时用等价分类法补充测试用例
3. 必要时再用猜错法补充测试用例
4. 若程序说明中含有输入条件组合，宜一开始就用因果图法，再按上述步骤进行

白盒测试 vs 黑盒测试

维度	白盒测试	黑盒测试
别名	结构测试、玻璃盒测试	功能测试、数据驱动测试
依据	程序内部逻辑结构	需求规格说明书
知晓内部结构	是（打开盒子）	否（黑盒子）
主要方法	逻辑覆盖/基本路径/循环覆盖	等价类/边界值/猜错法/因果图
适用阶段	早期（单元测试为主）	后期（确认/系统测试）
优势	测试路径精确，覆盖率可度量	贴近用户视角，不受实现影响
局限	不能发现遗漏的功能	无法保证内部结构覆盖

5 单元测试 (Unit Testing)

[p.66–78]

概述

[p.66]

单元测试（又称模块测试/分调）是动态测试的第一步，在编码阶段进行。测试最小独立编译单元——模块，在源码经过编译和评审确认无语法错误后开始。

测试策略

[p.67]

白盒法与黑盒法结合：先用黑盒法设计基本测试用例，再用白盒法按覆盖标准补充。单元测试以白盒法为主。

五大测试重点

[p.69–74]

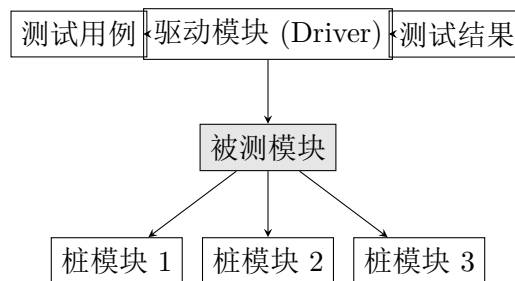
1. **模块接口：**实际输入与定义输入是否一致（个数、类型、顺序）；全局变量定义在各模块是否一致；外部资源使用是否检查可用性和释放。[p.70]

2. **局部数据结构**：变量未使用、错误变量名、不一致说明、未初始化、错误类型转换、数组越界、非法指针、拼写错误。[p.71]
3. **重要执行通路**：死代码、计算优先级错误、精度错误、比较运算错误（>, ≥, =, ==, ≠）、循环变量使用错误。[p.72]
4. **出错处理通路**：是否检查错误出现、资源使用前后检查、错误处理是否有效、报告记录的真实详细性。[p.73]
5. **边界条件**：普通合法/非法数据、边界内最近合法数据、边界外最近非法数据。[p.74]

驱动模块与桩模块

[p.78]

- **驱动模块 (Driver)**：接收测试数据，传给被测模块，打印结果
- **桩模块 (Stub)**：模拟实际被调用模块，完成少量数据处理，检验打印入口信息，返回控制



6 集成测试 (Integration Testing)

[p.79-93]

概述

[p.79]

集成测试（又称组装测试/联调）在单元测试完成后，将所有模块按概要设计要求组装成系统时进行。主要目标是发现与接口有关的问题。具有组装和检验双重意义。

测试内容

[p.80]

重点检查模块接口和全局数据：

- 穿越模块接口的数据是否丢失
- 一个模块是否对另一个模块产生不利副作用
- 模块组装后能否达到预期功能
- 全局数据结构引用是否有问题
- 模块误差积累是否可以接受

集成测试策略对比

[p.82-92]

维度	非渐增式	自顶向下渐增式	自底向上渐增式
策略	所有模块一次性组装测试	从主控模块开始，沿控制层次自上而下集成	从底层模块开始，逐步向上组合为功能簇
驱动/桩模块	大量驱动和桩模块	需要桩模块，可用实际模块作驱动	需要驱动模块，不需桩模块
错误定位	困难（模块一次性引入过多）	较易，每次只新增一个模块	较易，每次只新增少量模块
早期轮廓	无法显示	能较早显示程序轮廓，增强信心	不能显示，需到最后模块加入
主要问题	适用小规模程序	底层测试需要桩模块充分模拟	驱动模块编写可能较复杂
组装顺序	无	深度优先或广度优先	从下而上

混合渐增式测试

[p.92]

对上层模块自顶向下测试（早显轮廓），对关键模块（输入/输出功能、重要功能/性能、含新算法的模块）采用自底向上组装。

回归测试 (Regression Testing)

[p.93]

模块组装后新的数据流/控制流可能使原本正常的功能出错，需对已通过测试的测试用例重新执行。三类回归测试用例：

1. 能测试软件所有功能的代表性测试用例
2. 针对可能受修改影响的功能的附加测试用例
3. 针对修改过的软件成分的测试用例

7 确认测试 (Validation Testing)

[p.94-98]

概述

[p.94]

确认测试又称有效性测试，验证软件功能和性能等特性是否符合软件需求规格说明书。在模拟环境下运用黑盒法进行。是开发单位组织的最后一次测试，交付用户前的重要准备。

测试内容

[p.95]

- 功能测试：验证所有功能需求
- 性能测试：验证性能指标
- 强度测试：压力/负载条件下的表现
- 配置复审：检查软件配置是否齐全正确

α 测试与 β 测试

[p.96–98]

维度	α 测试 (Alpha Testing)	β 测试 (Beta Testing)
测试者	一个用户在开发环境（或机构内部模拟环境）	多个最终用户在实际使用场所
开发者参与	开发者在场” 指导”，记录错误	开发者通常不在测试现场
环境控制	受控环境	非受控真实环境
测试目标	评价功能 (FLURPS)、界面和特色	测试可支持性（文档、培训等）
启动时机	编码结束/模块测试完成/确认测试中产品稳定后	仅在 α 测试达到一定可靠程度后
错误记录	开发者负责记录	用户负责记录，定期向开发者报告

8 系统测试 (System Testing)

[p.99–104]

系统测试将经过确认测试的软件与硬件、外设、支持软件、数据、人员等元素结合，在实际运行环境下进行集成和确认测试。通常由任务委托单位或用户组织的验收小组负责。[p.99]

五种系统测试

[p.100–104]

测试类型	目的	方法
恢复测试 (Recovery)	验证系统故障后能否正常恢复	强制发生故障, 检验自动/手动恢复的正确性和修复时间
安全测试 (Security)	验证保护机制能否防非法侵入	攻击薄弱部分、引发系统出错、获取密码、浏览非保密数据
可用性测试 (Usability)	检查使用方便性、易理解性、易学性	检查用户界面、出错信息、译本、输入容错、输出一致性
安装测试 (Installation)	找出安装过程中的问题	检查硬件配置、选项方案相容性、所需文件、各部分齐全性
互连测试 (Interconnection)	验证不同系统间的互操作性	对支持标准规格说明或承诺互连的软件系统进行验证

9 调试 (Debugging)

[p.105-112]

概述

[p.105]

调试（纠错/排错）是程序测试后开始的工作，依据测试发现的错误迹象确定错误位置和原因并加以改正。两个组成部分：

1. 确定程序中可疑错误的确切性质和位置
2. 修改程序（设计、编码），排除错误

调试步骤

[p.106]

1. 从错误外部表现入手，确定程序中出错位置
2. 研究有关部分程序，找出错误内在原因
3. 修改设计和代码以排除错误
4. 重复原始测试，确认错误已排除且未引入新错误
5. 若修正无效则撤消改动，重复上述过程

调试难点

[p.107]

- 错误症状和原因可能相隔很远（尤其在高度耦合程序中）
- 错误症状可能在另一错误纠正后消失

- 错误症状可能并非由错误引起（如舍入误差）
- 可能由不易跟踪的人工操作引起
- 可能与时间相关
- 很难再现产生错误症状的输入条件
- 错误症状可能时有时无
- 可能由多处理器任务分布造成

四种调试策略对比

[p.108–111]

方法	基本思想	实现方式	适用场景
猜 测 法 (Guessing)	凭经验猜测错误原因并定位	插 入 打 印 语 句、注 释/GOTO、调试工具	经验丰富者快速定位
跟 踪 法 (Backtrack- ing)	从错误症状处沿控制流逆 向追踪源码	人工回溯程序控制流程	小程序高效，大程序回溯 路径多
演绎法 (De- duction)	列举可能原因 → 逐个排 除 → 验证剩余假设 → 纠 错	假设验证法（列原因 → 排 不可能 → 析剩余 → 证明 假设）	逻辑条理清晰的问题
归纳法 (In- duction)	从线索（错误征兆）分析 关系找出错误	收集数据 → 组织 → 研究 关系 → 提出假设 → 证明 → 纠错	有规律可循的错误

调试原则

[p.112]

1. **冷静多思**：调试是最艰巨的脑力劳动之一，错误症状与原因间无明显联系，需要耐心细心，必要时虚心请教同行
2. **彻底修改**：注意错误群集现象（周围可能存在类似错误），纠正错误本身而非仅消除症状
3. **防止引入新错误**：修改后进行回归测试

10 小结 (Summary)

[p.113]

1. **测试根本任务**：发现软件错误。软件错误往往出现在分析和设计阶段，发现越晚改正代价越大
2. **静态测试关键**：开发前期利用静态测试（代码复审）非常重要，发现错误效率往往更高
3. **测试与调试交替**：测试发现错误症状，调试定位和改正原因，两者交替进行
4. **测试的局限性**：测试只能说明软件**存在错误**，不能证明软件**没有错误**——任何经过测试的软件都不能保证不再存在错误

5. **投入比例：**一般软件开发组织将 30%–40% 项目精力投入测试，关键软件（如飞行控制）测试费用更高

表 1: *
本章核心概念中英对照表

中文	English
软件测试	Software Testing
白盒测试 / 结构测试	White-Box Testing / Structural Testing
黑盒测试 / 功能测试	Black-Box Testing / Functional Testing
逻辑覆盖	Logic Coverage
语句覆盖	Statement Coverage
判定覆盖	Decision Coverage (Branch Coverage)
条件覆盖	Condition Coverage
判定-条件覆盖	Decision/Condition Coverage
条件组合覆盖	Multiple Condition Coverage
基本路径测试	Basis Path Testing
循环复杂性（环形复杂度）	Cyclomatic Complexity
循环覆盖	Loop Coverage
等价类划分	Equivalence Partitioning
边界值分析	Boundary Value Analysis
猜错法	Error Guessing
因果图法	Cause-Effect Graphing
判定表	Decision Table
单元测试	Unit Testing
驱动模块	Driver Module
桩模块	Stub Module
集成测试	Integration Testing
回归测试	Regression Testing
确认测试	Validation Testing
Alpha/Beta 测试	Alpha/Beta Testing
系统测试	System Testing
恢复测试	Recovery Testing
安全性测试	Security Testing
可用性测试	Usability Testing
调试	Debugging
代码复审	Code Review
代码会审	Code Inspection
走查	Walkthrough
V 模型	V-Model
Pareto 原则	Pareto Principle

来源：软件工程基础第 8 章软件测试（中南大学任胜兵）

113 页课件全部覆盖 • 2026 年 6 月 1 日